Платформено-независими програмни езици

Лекция 7

Изключения и вход/изход

Обработка на грешки, собствени изключения, try-with-resources, потоци за вход/изход, файлове с NIO.2, CSV и JSON



проф. д-р Даниела Гоцева
Учебна 2026/2027 г.

Съдържание на лекцията

1. Грешки по време на изпълнение
2. Изключения в Java
3. Генериране и собствени изключения
4. try-with-resources
5. Вход/изход: потоци
6. Файлове с NIO.2
7. Формати на данните

01

Грешки по време на изпълнение

Какво може да се обърка и как да реагираме

Видове грешки

- **Синтактични** – откриват се от компилатора
- **Логически** – програмата работи, но дава грешен резултат
- **Грешки по време на изпълнение** (run-time):
  - липсващ файл, прекъсната мрежова връзка
  - невалиден вход от потребителя
  - деление на нула, индекс извън масива
  - недостиг на памет
- Някои са **предвидими** и трябва да се обработят, други са **бъгове** и трябва да се поправят

Как да сигнализираме за грешка?

Код за грешка (стар подход, C)

- функцията връща `-1`, `null`, `false`
- извикващият **може да забрави** да провери
- кодът за обработка е смесен с основната логика
- грешката трябва ръчно да се предава нагоре

Изключения (Java, Python)

- грешката „прекъсва“ нормалния ход
- **не може** да бъде игнорирана незабелязано
- обработката е отделена в `catch`
- изключението се предава нагоре автоматично

Предимства на изключенията

- **Разделяне** на нормалната логика от обработката на грешки
- **Разпространяване** нагоре по стека на извикванията до мястото, което може да реагира
- **Групиране** и класифициране на грешките по тип (йерархия от класове)
- **Информация** за грешката: съобщение, причина, стек на извикванията
- Познато от Python: `try` / `except` / `finally` / `raise`

Изключения в Python и в Java

Python

- `try:/except ValueError as e:`
- `finally:`
- `raise ValueError("...")`
- `with open(...) as f:`
- всички изключения са „непроверими“

Java

- `try { }/catch (NumberFormatException e) { }`
- `finally { }`
- `throw new IllegalArgumentException("...")`
- `try (var r = Files.newBufferedReader(p)) { }`
- проверимите изключения се контролират от компилатора

02

Изключения в Java

Йерархия, проверими и непроверими изключения, try-catch-finally

Йерархия на изключенията

- `Throwable` – корен на всички изключения
  - `Error` – сериозни проблеми на JVM: `OutOfMemoryError`, `StackOverflowError`
    - не се обработват от приложението
  - `Exception` – грешки, които програмата може да обработи
    - `IOException`, `SQLException`... – **провери**
    - `RuntimeException` – **непроверими**
      - `NullPointerException`, `IllegalArgumentException`, `ArithmeticException`...

Проверими и непроверими изключения

Проверими (checked)

- наследници на `Exception`, но не на `RuntimeException`
- компилаторът **изисква** да ги обработим или декларираме с `throws`
- външни проблеми: файлове, мрежа, база данни
- `IOException`, `SQLException`

Непроверими (unchecked)

- наследници на `RuntimeException`
- компилаторът **не изисква** обработка
- обикновено бъгове в програмата
- `NullPointerException`, `IndexOutOfBoundsException`

Често срещани изключения

Изключение	Кога възниква
`NullPointerException`	извикване на метод върху `null`
`ArrayIndexOutOfBoundsException`	индекс извън масива
`ArithmeticException`	целочислено деление на нула
`NumberFormatException`	`Integer.parseInt("abc")`
`ClassCastException`	грешно преобразуване на тип
`IllegalArgumentException`	невалиден аргумент на метод
`IllegalStateException`	методът е извикан в неподходящ момент
`IOException`, `FileNotFoundException`	грешка при вход/изход

try и catch

- `try` – кодът, който може да предизвика изключение
- `catch` – обработка на конкретен тип изключение
- Проверяват се **по ред** – първо по-специфичните
- Изпълнява се първият съвпадащ `catch`

```
Scanner in = new Scanner(System.in);
try {
    System.out.print("Число: ");
    int n = Integer.parseInt(in.nextLine());
    System.out.println(100 / n);
} catch (NumberFormatException e) {
    System.out.println("Това не е число!");
} catch (ArithmeticException e) {
    System.out.println("Деление на нула!");
}
System.out.println("Край");
```

Multi-catch и finally

- Един `catch` за няколко типа: `A | B`
- `finally` – изпълнява се **винаги**: с или без изключение, дори при `return`
- Използва се за освобождаване на ресурси

```
try {
    process(data);
} catch (IOException | SQLException e) {
    System.err.println("Грешка: "
            + e.getMessage());
} finally {
    System.out.println("Почистване...");
}
```

Ход на изпълнение

Ситуация	Какво се изпълнява
няма изключение	целият `try` → `finally` → кодът след блока
изключение, обработено от `catch`	`try` до грешката → съответният `catch` → `finally` → кодът след блока
изключение, необработено тук	`try` до грешката → `finally` → изключението продължава нагоре
`return` в `try`	`finally` се изпълнява преди връщането

Стек на извикванията (stack trace)

- Показва **къде** е възникнало изключението и през кои методи е минало
- Чете се отгоре надолу: първият ред е мястото на грешката
- `e.printStackTrace()` – за отстраняване на грешки
- В реални програми – в лог (лекция 9)

```
Exception in thread "main"
  java.lang.ArithmeticException: / by zero
    at Calc.divide(Calc.java:12)
    at Calc.average(Calc.java:20)
    at Main.main(Main.java:7)
```

Пример: повторно въвеждане до коректна стойност

- Изключението от `parseInt` се използва за проверка на входа
- Цикълът продължава, докато входът стане коректен
- Потребителят получава ясно съобщение

```
static int readInt(Scanner in, String prompt,
                   int min, int max) {
    while (true) {
        System.out.print(prompt);
        try {
            int v = Integer.parseInt(
                    in.nextLine().strip());
            if (v >= min && v <= max) {
                return v;
            }
            System.out.println("Между " + min
                    + " и " + max + "!");
        } catch (NumberFormatException e) {
            System.out.println("Не е число!");
        }
    }
}
```

03

Генериране и собствени изключения

throw, throws, свързани изключения

Генериране на изключение: throw

- `throw` + обект-изключение
- Прекъсва изпълнението на метода
- Използвайте най-подходящия стандартен тип
- Описателно съобщение – какво и защо

```
public void withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Сумата трябва да е положителна: "
            + amount);
    }
    if (amount > balance) {
        throw new IllegalStateException(
            "Недостатъчна наличност");
    }
    balance -= amount;
}
```

Деклариране: throws

- Метод, който може да хвърли **проверимо** изключение, трябва да го декларира
- Извикващият или го обработва, или също го декларира
- Така изключението се предава нагоре по стека

```
public static List<String> load(String file)
        throws IOException {
    return Files.readAllLines(Path.of(file));
}

public static void main(String[] args) {
    try {
        List<String> lines = load("data.txt");
        System.out.println(lines.size());
    } catch (IOException e) {
        System.err.println("Не мога да прочета: "
                + e.getMessage());
    }
}
```

Собствен клас за изключение

- Наследява `Exception` (проверимо) или `RuntimeException` (непроверимо)
- Име, завършващо на `Exception`
- Може да носи допълнителна информация

```
public class InsufficientFundsException
        extends Exception {

    private final double missing;

    public InsufficientFundsException(
            double missing) {
        super("Липсват " + missing + " лв.");
        this.missing = missing;
    }

    public double getMissing() {
        return missing;
    }
}
```

Използване на собственото изключение

- Методът декларира `throws`
- Извикващият получава **смислен тип** и допълнителни данни

```
public void withdraw(double amount)
        throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException(
                amount - balance);
    }
    balance -= amount;
}

try {
    account.withdraw(500);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage());
    System.out.println("Внесете поне "
            + e.getMissing());
}
```

Свързани изключения (chained)

- Обвиваме ниско ниво изключение в изключение на по-високо ниво
- Оригиналът се пази като **причина** (cause)
- `getCause()` – достъп до причината
- Не губим информация за първоначалната грешка

```
public Config loadConfig(Path p) {
    try {
        String text = Files.readString(p);
        return Config.parse(text);
    } catch (IOException e) {
        throw new ConfigException(
            "Конфигурацията не може да се "
            + "зареди: " + p, e);   // причина
    }
}

// В stack trace:
// ConfigException: Конфигурацията...
// Caused by: java.nio.file.NoSuchFileException
```

Повторно хвърляне и обработка на ниво

- Методът може да хване, да запише и да **хвърли отново**
- Обработката се прави на нивото, което **знае какво да прави**
- Ниските нива – съобщават; горните – решават

```
void importFile(Path p) throws IOException {
    try {
        List<String> lines = Files.readAllLines(p);
        save(lines);
    } catch (IOException e) {
        log.warning("Импортът неуспешен: " + p);
        throw e;                    // нагоре
    }
}

// Горно ниво — потребителски интерфейс
try {
    importFile(Path.of("data.csv"));
} catch (IOException e) {
    showError("Файлът не може да бъде прочетен");
}
```

Изключения в ламбда изрази

- Стандартните функционални интерфейси **не позволяват** проверими изключения
- `Files.readString` в `map` → грешка при компилиране
- Решение: обвиване в непроверимо изключение
- `UncheckedIOException` – стандартна обвивка за `IOException`

```
List<Path> files = List.of(
        Path.of("a.txt"), Path.of("b.txt"));

List<String> texts = files.stream()
    .map(p -> {
        try {
            return Files.readString(p);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    })
    .toList();
```

Изключения и наследяване

- Припокрит метод **не може** да хвърля по-общи проверими изключения от оригинала
- Може да хвърля същите, по-специфични или никакви
- Причина: кодът, работещ с родителя, трябва да е подготвен за всички изключения
- Непроверимите изключения нямат такова ограничение
- `catch (Exception e)` хваща и всички наследници – затова специфичните `catch` са първи

Добри и лоши практики

Правилно

- хващайте **конкретни** типове
- обработвайте там, където можете да реагирате смислено
- съобщение с контекст: какво, къде, защо
- запазвайте причината при обвиване
- валидирайте входа рано

Грешно

- празен `catch { }` – „поглъщане“ на грешката
- `catch (Exception e)` навсякъде
- изключения за нормален контрол на потока
- `e.printStackTrace()` в продукционен код
- загубена причина при обвиване

04

try-with-resources

Автоматично затваряне на ресурси (Java 7+)

Затваряне на ресурси: преди и сега

C finally (преди Java 7)

- отваряме ресурса преди `try`
- в `finally` проверяваме за `null`
- самото `close()` хвърля изключение → вложен `try`
- лесно се забравя → изтичане на ресурси

try-with-resources

- ресурсът се декларира в `try ( ... )`
- затваря се **автоматично**
- работи с всеки `AutoCloseable`
- няколко ресурса – затварят се в обратен ред

try-with-resources – пример

- Ресурсът е видим само в `try` блока
- `close()` се извиква винаги – и при изключение
- Може без `catch` – ако методът декларира `throws`

```
Path in = Path.of("input.txt");
Path out = Path.of("output.txt");

try (BufferedReader r = Files.newBufferedReader(in);
     BufferedWriter w = Files.newBufferedWriter(out)) {

    String line;
    while ((line = r.readLine()) != null) {
        w.write(line.toUpperCase());
        w.newLine();
    }
} catch (IOException e) {
    System.err.println("Грешка: " + e.getMessage());
}
```

Файлове в Python и в Java

Python

- `with open("a.txt", encoding="utf-8") as f:`
- `for line in f:`
- `f.read()` – целия файл
- `json.load(f)` – вграден модул за JSON
- `except FileNotFoundError:`

Java

- `try (var r = Files.newBufferedReader(p)) {`
- `Files.lines(p).forEach(...)`
- `Files.readString(p)` – целия файл
- JSON – външна библиотека (Jackson, Gson)
- `catch (NoSuchFileException e)`

Собствен ресурс: AutoCloseable

- Имплементираме `AutoCloseable` с метод `close()`
- Може да се използва в `try-with-resources`
- Подходящо за връзки, файлове, сесии, заключвания

```
public class Connection
        implements AutoCloseable {

    public Connection(String host) {
        System.out.println("Свързване " + host);
    }
    public void send(String msg) {
        System.out.println("> " + msg);
    }
    @Override public void close() {
        System.out.println("Затваряне");
    }
}

try (var c = new Connection("srv")) {
    c.send("Здравей");
}
// Свързване srv  > Здравей  Затваряне
```

Потиснати изключения (suppressed)

- Ако и тялото, и `close()` хвърлят изключение – кое да видим?
- try-with-resources запазва **първото** (от тялото)
- Изключението от `close()` е **потиснато** – прикачено към първото
- `getSuppressed()` – достъп до тях

```
try (var c = new Connection("srv")) {
    throw new IllegalStateException("в тялото");
} catch (Exception e) {
    System.out.println(e.getMessage());
    for (Throwable s : e.getSuppressed()) {
        System.out.println("потиснато: "
                + s.getMessage());
    }
}
```

05

Вход/изход: потоци

Байтови и символни потоци, буфериране, кодировки

Семантика на потоците за вход/изход

- **Поток** (stream) – последователност от данни от **източник** към **приемник**
- Източници/приемници: файл, конзола, мрежа, памет
- Програмата **чете** от входен поток и **пише** в изходен
- Еднаква работа, независимо от устройството
- Потоците трябва да се **затварят**
- Не бъркайте с `java.util.stream.Stream` от миналата лекция!

Йерархия на класовете за вход/изход

	Вход	Изход
Байтове (двоични данни)	`InputStream`	`OutputStream`
– файл	`FileInputStream`	`FileOutputStream`
– буфериран	`BufferedInputStream`	`BufferedOutputStream`
– примитивни типове	`DataInputStream`	`DataOutputStream`
Символи (текст)	`Reader`	`Writer`
– файл	`FileReader`	`FileWriter`
– буфериран	`BufferedReader`	`BufferedWriter`, `PrintWriter`
– мост байтове ↔ символи	`InputStreamReader`	`OutputStreamWriter`

Обвиване на потоци (декоратори)

- Потоците се „обвиват“ един в друг
- Всеки слой добавя възможност: буфер, кодировка, типове
- Шаблонът **Decorator**
- Буферирането намалява броя на достъпите до диска

```
InputStream raw =
        new FileInputStream("data.txt");
Reader chars =
        new InputStreamReader(raw,
                StandardCharsets.UTF_8);
BufferedReader buffered =
        new BufferedReader(chars);

String first = buffered.readLine();
buffered.close();   // затваря всички слоеве

// По-кратко с NIO.2:
BufferedReader r = Files.newBufferedReader(
        Path.of("data.txt"));
```



Винаги задавайте кодировката: UTF-8

Иначе кирилицата може да се превърне в `????` или `Ð¿Ñ€Ð¸`. От Java 18 UTF-8 е кодировка по подразбиране, но явното задаване прави кода ясен и преносим.

Конзолата: System.in, out, err

- `System.out` – стандартен изход
- `System.err` – изход за грешки
- `System.in` – стандартен вход (байтов поток)
- `Scanner` – удобно четене и разбор на текст

```
Scanner sc = new Scanner(System.in);

System.out.print("Име: ");
String name = sc.nextLine();

System.out.print("Възраст: ");
int age = sc.nextInt();

System.out.printf("%s е на %d г.%n",
        name, age);

if (age < 0) {
    System.err.println("Невалидна възраст");
}
```

Разделяне на текст на части (tokenizing)

- `String.split` – по регулярен израз
- `Scanner` – чете числа и думи от низ или файл
- `useDelimiter` – собствен разделител
- `StringTokenizer` – остарял клас

```
String line = "Иван;6;5;4";
String[] parts = line.split(";");
// [Иван, 6, 5, 4]

Scanner s = new Scanner("12 7 30 5");
int sum = 0;
while (s.hasNextInt()) {
    sum += s.nextInt();
}
// sum = 54

Scanner csv = new Scanner("a,b,,c")
        .useDelimiter(",");
```

Scanner върху файл

- `Scanner` чете и от файл, не само от клавиатурата
- Удобен за числа, разделени с интервали или нови редове
- Задайте кодировката

```
Path p = Path.of("numbers.txt");

try (Scanner sc = new Scanner(p,
        StandardCharsets.UTF_8)) {
    double sum = 0;
    int count = 0;
    while (sc.hasNextDouble()) {
        sum += sc.nextDouble();
        count++;
    }
    System.out.printf("Средно: %.2f%n",
            sum / count);
}
```

06

Файлове с NIO.2

Path и Files – съвременният начин (Java 7+)

Path – път до файл или папка

- `Path.of(...)` – създава път
- Не проверява дали файлът съществува
- Методи за работа с пътя: `getFileName`, `getParent`, `resolve`
- Работи еднакво под Windows, macOS и Linux

```
Path p = Path.of("data", "students.csv");
System.out.println(p);  // data/students.csv

p.getFileName();          // students.csv
p.getParent();            // data
p.toAbsolutePath();       // /home/.../data/...

Path home = Path.of(
        System.getProperty("user.home"));
Path docs = home.resolve("Documents");

Files.exists(p);          // true / false
Files.isDirectory(docs);
```

Четене на текстови файлове

- `readString` – целият файл като низ (Java 11)
- `readAllLines` – списък от редове
- `lines` – **поток** от редове, мързеливо – за големи файлове
- `lines` трябва да се затвори → `try-with-resources`

```
Path p = Path.of("students.csv");

String all = Files.readString(p);

List<String> lines = Files.readAllLines(p);

try (Stream<String> s = Files.lines(p)) {
    long excellent = s
        .skip(1)                          // заглавие
        .map(l -> l.split(","))
        .filter(f ->
            Double.parseDouble(f[2]) >= 5.5)
        .count();
}
```

Запис в текстови файлове

- `writeString` – записва низ
- `write` – записва списък от редове
- Опции: `APPEND`, `CREATE`, `TRUNCATE_EXISTING`
- `newBufferedWriter` – за постепенен запис

```
Path out = Path.of("report.txt");

Files.writeString(out, "Отчет\n");

Files.write(out,
        List.of("ред 1", "ред 2"),
        StandardOpenOption.APPEND);

try (BufferedWriter w =
        Files.newBufferedWriter(out)) {
    for (int i = 1; i <= 3; i++) {
        w.write("Ред " + i);
        w.newLine();
    }
}
```

PrintWriter – форматиран изход във файл

- Методи `print`, `println`, `printf` – като `System.out`
- Удобен за отчети и таблици
- Не хвърля `IOException` – проверете с `checkError()`

```
List<Student> list = loadStudents();

try (PrintWriter pw = new PrintWriter(
        Files.newBufferedWriter(
            Path.of("report.txt")))) {

    pw.printf("%-10s %6s%n", "Име", "Оценка");
    pw.println("-".repeat(17));
    for (Student s : list) {
        pw.printf("%-10s %6.2f%n",
                  s.name(), s.grade());
    }
}
```

Двоични файлове: копиране

- `InputStream.transferTo` – копира всички байтове (Java 9)
- Ръчно копиране с буфер – класическият начин
- `Files.copy` – най-просто

```
Path src = Path.of("photo.jpg");
Path dst = Path.of("copy.jpg");

try (InputStream in = Files.newInputStream(src);
     OutputStream out =
             Files.newOutputStream(dst)) {
    byte[] buf = new byte[8192];
    int n;
    while ((n = in.read(buf)) != -1) {
        out.write(buf, 0, n);
    }
}

Files.copy(src, dst,
        StandardCopyOption.REPLACE_EXISTING);
```

Операции с файлове и папки

Метод	Действие
`Files.exists(p)`, `isDirectory(p)`	проверка за съществуване / вид
`Files.createDirectories(p)`	създава папка (и родителските)
`Files.copy(a, b)`, `Files.move(a, b)`	копиране / преместване
`Files.delete(p)`, `deleteIfExists(p)`	изтриване
`Files.size(p)`, `getLastModifiedTime(p)`	размер, време на промяна
`Files.list(dir)`	поток от съдържанието на папка
`Files.walk(dir)`	поток от цялото дърво от файлове
`Files.createTempFile(...)`	временен файл

Обхождане на папки

- `Files.list` – само преките наследници
- `Files.walk` – рекурсивно цялото дърво
- Връщат потоци → `try-with-resources`

```
Path dir = Path.of("src");

try (Stream<Path> files = Files.walk(dir)) {
    files.filter(f -> f.toString()
                       .endsWith(".java"))
         .forEach(System.out::println);
}

try (Stream<Path> files = Files.walk(dir)) {
    long total = files
        .filter(Files::isRegularFile)
        .mapToLong(f -> f.toFile().length())
        .sum();
    System.out.println(total + " байта");
}
```

Конфигурационни файлове: Properties

- Текстов формат `ключ=стойност`
- Класът `Properties` чете и записва такива файлове
- Удобен за настройки: адрес на сървър, порт, език

```
// app.properties
// server.host=localhost
// server.port=8080

Properties cfg = new Properties();
try (Reader r = Files.newBufferedReader(
        Path.of("app.properties"))) {
    cfg.load(r);
}

String host = cfg.getProperty("server.host");
int port = Integer.parseInt(
    cfg.getProperty("server.port", "80"));
```

07

Формати на данните

Двоични данни, сериализация, CSV и JSON

Текстови и двоични формати

Текстови (CSV, JSON, XML)

- четими от човек
- преносими между езици и системи
- по-големи по обем
- нужен е разбор (parsing)

Двоични

- компактни и бързи
- не се четат от човек
- зависят от формата и реда на байтовете
- изображения, архиви, собствени формати

Двоични данни: DataStream

- `DataOutputStream` – записва примитивни типове в двоичен вид
- `DataInputStream` – чете ги в **същия ред**
- Независимо от платформата (big-endian)

```
Path f = Path.of("grades.bin");

try (var out = new DataOutputStream(
        new BufferedOutputStream(
            Files.newOutputStream(f)))) {
    out.writeInt(3);
    out.writeDouble(5.5);
    out.writeUTF("Ана");
}

try (var in = new DataInputStream(
        new BufferedInputStream(
            Files.newInputStream(f)))) {
    int n = in.readInt();
    double g = in.readDouble();
    String name = in.readUTF();
}
```

Сериализация на обекти

- Обект → поток от байтове → обект
- Класът имплементира `Serializable`
- `transient` – полето не се записва
- **Внимание:** десериализация на чужди данни е риск за сигурността
- Днес за обмен на данни се предпочита **JSON**

```
record Student(String name, double grade)
        implements Serializable { }

Path f = Path.of("st.ser");
try (var out = new ObjectOutputStream(
        Files.newOutputStream(f))) {
    out.writeObject(
        new Student("Ана", 5.5));
}

try (var in = new ObjectInputStream(
        Files.newInputStream(f))) {
    Student s = (Student) in.readObject();
}
```

Пример: CSV файл → записи

```
record Student(String name, int fn, double grade) {
    static Student parse(String line) {
        String[] f = line.split(",");
        return new Student(f[0].strip(), Integer.parseInt(f[1].strip()),
                           Double.parseDouble(f[2].strip()));
    }
}

List<Student> students;
try (Stream<String> lines = Files.lines(Path.of("students.csv"))) {
    students = lines.skip(1)                          // заглавен ред
                    .filter(l -> !l.isBlank())
                    .map(Student::parse)
                    .toList();
}

double avg = students.stream().mapToDouble(Student::grade).average().orElse(0);
```

`Files.lines` + `record` + Stream API – типичен съвременен начин за обработка на данни

Грешни редове във входните данни

- Реалните файлове съдържат грешки
- Не спираме цялата обработка заради един ред
- Събираме грешките и ги докладваме накрая

```
List<Student> ok = new ArrayList<>();
List<String> bad = new ArrayList<>();
int n = 0;

for (String line : Files.readAllLines(p)) {
    n++;
    try {
        ok.add(Student.parse(line));
    } catch (RuntimeException e) {
        bad.add("ред " + n + ": "
                + e.getMessage());
    }
}
System.out.println("Прочетени: " + ok.size());
bad.forEach(System.err::println);
```

JSON с библиотеката Jackson

```java
// Maven зависимост: com.fasterxml.jackson.core:jackson-databind

record Student(String name, int fn, double grade) { }

ObjectMapper mapper = new ObjectMapper();

// Обект -> JSON
String json = mapper.writeValueAsString(new Student("Ана", 101, 5.5));
// {"name":"Ана","fn":101,"grade":5.5}

// JSON -> обект
Student s = mapper.readValue(json, Student.class);

// Списък от файл
List<Student> all = mapper.readValue(Path.of("students.json").toFile(),
        new TypeReference<List<Student>>() { });
```

JSON е основният формат за обмен на данни в уеб и при работа с ИИ услуги (лекция 10)

Кой клас за вход/изход да изберем?

Задача	Препоръка
малък текстов файл наведнъж	`Files.readString`, `Files.readAllLines`
голям текстов файл ред по ред	`Files.lines` или `Files.newBufferedReader`
запис на текст	`Files.writeString`, `Files.newBufferedWriter`, `PrintWriter`
числа и думи от текст	`Scanner`
двоични данни	`Files.newInputStream` / `newOutputStream` + буфер
структурирани данни за обмен	JSON (Jackson), CSV
операции с файлове и папки	`Files.copy`, `move`, `delete`, `walk`

Обобщение

- Изключенията отделят обработката на грешки от основната логика
- `Error` / проверими `Exception` / непроверими `RuntimeException`
- `try` – `catch` – `finally`; multi-catch; `throw` и `throws`
- Собствени изключения и свързване с причина (cause)
- **try-with-resources** – автоматично затваряне на `AutoCloseable`
- Байтови и символни потоци, декоратори, буфериране, UTF-8
- NIO.2: `Path` и `Files` – четене, запис, обхождане на папки
- Формати: двоични, сериализация, CSV, JSON

Въпроси за самопроверка

- Каква е разликата между проверимо и непроверимо изключение?
- Кога се изпълнява блокът `finally`?
- Защо празният `catch` блок е лоша практика?
- Какво е свързано изключение и защо пазим причината?
- Какво трябва да имплементира клас, за да се използва в try-with-resources?
- Каква е разликата между `Files.readAllLines` и `Files.lines`?
- Защо е важно да задаваме кодировката при четене на текст?

Въпроси?

Следваща лекция: Нишки и мрежово програмиране с TCP

